AEC6 - Flujo de lectura en Cassandra

Introducción

En esta actividad he trabajado el flujo de lectura de datos en Cassandra, centrándome en cómo funciona el Flujo de lectura (*Read Path Flow)* cuando se realiza una consulta de lectura. Con esta práctica he intentado entender qué ocurre realmente dentro de Cassandra dependiendo del estado de sus estructuras internas y de lo que ha pasado antes en el sistema.

Para ello, he planteado tres escenarios de lectura distintos, cada uno con un contexto diferente. La idea es mostrar que Cassandra no sigue siempre el mismo camino para resolver una lectura, sino que adapta el flujo en función de si los datos están en memoria, en caché o únicamente en disco.

Antes de entrar en los escenarios de lectura, he incluido una breve explicación de los conceptos más básicos necesarios para entender el flujo de lectura en Cassandra y las estructuras que intervienen en él. Esta parte sirve como contexto general y para poder seguir mejor los escenarios posteriores y ha servido de guía para mí durante el desarrollo de la práctica.

¿Por qué Cassandra?

Cassandra es un referente como base de datos distribuida pensada para trabajar con grandes volúmenes de datos y altas cargas de lectura y escritura. A diferencia de bases de datos relacionales tradicionales, Cassandra como hemos visto está diseñada desde el principio para escalar horizontalmente a lo bestia y seguir funcionando incluso cuando fallan nodos del sistema.

Desde mi punto de vista, Cassandra es especialmente interesante porque muchas de sus decisiones internas no son tann evidentes si solo se ve desde fuera. Una simple consulta de lectura puede implicar varias estructuras diferentes, tanto en memoria como en disco, y entender ese proceso sobre todo viniendo de otros proyectos y asignaturas de bajo nivel creo que ayuda a comprender por qué Cassandra ofrece buen rendimiento incluso con millones de registros.

¿Qué tiene esta BD que afecta directamente a las lecturas?

Una de las claves de Cassandra para mi es su arquitectura de almacenamiento donde:

- Las escrituras se realizan primero en memoria.
- Los datos se vuelcan posteriormente a disco en unos archivos llamados SSTables (que enncima son inmutables).
- Con el tiempo, los datos pueden quedar repartidos entre varios de estos archivos.

Esto hace que el proceso de lectura sea bastante más complejo que en otros sistemas, ya que Cassandra tiene que localizar y combinar datos que pueden estar en diferentes lugares. Precisamente por eso, el flujo de lectura está tan optimizado.

> Este diseño a mi parecer prioriza la velocidad de escritura, y el flujo de lectura existe para compensar la complejidad que eso introduce en las lecturas.

El Flujo de Lectura (Read Path Flow)

¿Qué es el Read Path Flow?

Voy a definir este concepto como el conjunto de pasos que Cassandra sigue para resolver una consulta de lectura. No se trata de un camino fijo, sino de una serie de comprobaciones que se realizan en un orden determinado, empezando por las estructuras más rápidas y avanzando hacia las más costosas en términos de latencia.

¿Por qué Cassandra no va directamente a por el disco?

Ir directamente al disco en cada lectura sería muy costoso en términos de rendimiento. Por eso, Cassandra intenta resolver la consulta utilizando antes estructuras en memoria o caché. De forma general, el flujo de lectura puede implicar:

- Memtable, donde pueden estar los datos escritos más recientemente.
- Row Cache, si está habilitada, que puede contener filas completas accedidas con frecuencia.
- Partition Key Cache, para evitar búsquedas repetidas en estructuras de índice.
- Bloom Filters, que permiten descartar rápidamente SSTables que seguro no contienen la clave buscada.
- Estructuras en disco como el Partition Summary, el Partition Index y el Compression Offset Map, que permiten localizar los datos de forma precisa dentro de una SSTable.
- Finalmente, la lectura de los datos desde la SSTable correspondiente.

¿Por qué hay tantas estructuras distintas?

No todas las estructuras del Read Path almacenan el mismo tipo de información, ni están pensadas para el mismo objetivo. Por ejemplo:

- La Memtable contiene datos recientes que aún no han sido volcados a disco.
- La Row Cache puede contener filas completas que han sido leídas con frecuencia, aunque sean datos antiguos.
- El Bloom Filter no almacena datos, solo indica si tiene sentido seguir buscando en una SSTable.
- El Partition Summary y el Partition Index sirven para navegar por las SSTables sin tener que leerlas enteras.

Desde mi punto de vista, esta separación de responsabilidades es lo que permite a Cassandra mantener un buen rendimiento incluso cuando el volumen de datos crece mucho.

> Un punto que me ha parecido importante es que el contenido de estas estructuras depende de lo que ha ocurrido antes en el sistema (lecturas, escrituras, flushes, etcc), y no solo de la consulta actual.

¿Qué quiero mostrar con los escenarios?

Para esta práctica he definido tres escenarios de lectura distintos, variando el uso de cachés y el estado de los datos. La intención no es repetir tres veces el mismo flujo, sino mostrar cómo pequeñas diferencias en el contexto provocan cambios importantes en el recorrido del Read Path.

En cada escenario explico:

- Qué datos existen y dónde están almacenados.
- Qué estructuras se consultan y cuáles se omiten.
- Por qué Cassandra toma una decisión concreta en cada paso.

He intentado que los ejemplos sean medianamente realistas, utilizando valores ficticios claramente pero coherentes, de forma que el flujo se pueda seguir con claridad y tenga sentido desde el punto de vista del funcionamiento interno de Cassandra.

En los siguientes apartados se describen los tres escenarios de lectura, comenzando por un caso en el que Cassandra puede resolver la consulta rápidamente gracias al uso de caché y avanzando hacia escenarios en los que es necesario recorrer completamente las estructuras en disco.

Escenario 1

Lectura resuelta mediante Row Cache (Partition Key Cache habilitada)

Con este primer escenario quiero mostrar el caso más eficiente posible dentro del flujo de lectura de Cassandra una lectura que se resuelve sin necesidad de acceder a disco gracias al uso de la Row Cache. La idea es entender cómo Cassandra puede devolver una fila completa directamente desde memoria cuando los datos han sido accedidos con frecuencia.

Este escenario sirve también como punto de partida para los siguientes, ya que permite comparar un flujo creo que medianamente rapido comparado con otros más costosos donde no se dispone de este tipo de caché.

Requisitos previos

Antes de que se realice la lectura, mi sistema se encuentra en la siguiente situación:

- Existe una tabla `users`.
- La fila con partition key `pk1` y clustering key `ck1` fue escrita hace tiempo y ya ha sido volcada a disco tras un flush de la tabla de memoria.
- Esa misma fila ha sido leída varias veces recientemente.

Debido a estos accesos repetidos, Cassandra ha almacenado la fila completa en la Row Cache, ya que se trata de un dato hot, es decir, accedido con frecuencia.

La Partition Key Cache está habilitada en el sistema, aunque en este escenario no será determinante, ya que la lectura se resuelve antes de necesitarla.

```
SELECT * FROM users WHERE pk = 'pk1' AND ck = 'ck1';
```

¿Dónde están los datos en este momento?

Antes de seguir el flujo, es importante dejar claro qué contiene cada estructura relevante:

- La tabla de memoria contiene datos escritos recientemente, pero no incluye `pk1`, ya que esta fila fue escrita hace tiempo y ya se encuentra en SSTables.
- Row Cache (habilitada) Contiene la fila completa

  ```
  {
    pk1: {
      ck1: { name: "Ismael", age: 21 }
    }
  }
  ```

Esta fila está en caché porque ha sido leída varias veces recientemente.

- Partition Key Cache (habilitada): Puede contener información sobre claves de partición, pero no llega a utilizarse en este escenario.
- SSTables: Contienen los datos persistidos en disco, pero no será necesario acceder a ellas.

Flujo de lectura

¿Se consulta la Memtable?

Sí, aquí cassandra comienza comprobando la Memtable para ver si existen datos más recientes que aún no se hayan volcado a disco.

Resultado: Miss y la clave `pk1` no se encuentra en la Memtable lo cual tiene sentido, ya que no se trata de una escritura reciente.

¿Qué ocurre con la Row Cache?

A continuación, Cassandra comprueba la Row Cache, que en este escenario está habilitada.

Resultado: Hit, ya que la Row Cache contiene la fila completa asociada a `pk1` y `ck1`, por lo que Cassandra puede devolver directamente los datos solicitados sin continuar el flujo de lectura. En este punto, el proceso termina.

```
Cliente -> Nodo Cassandra
  |-> Memtable: Miss (pk1 no presente)
  |-> Row Cache: Hit
      {pk1: {ck1: {name: "ismael", age: 21}}}
  |-> Resultado devuelto al cliente
```

¿Por qué no se consultan el resto de estructuras?

En este escenario no se accede a Bloom Filter, Partition Key Cache, Partition Summary, Partition Index, Compression Offset Map ni a las SStables.

La razón es sencilla la Row Cache devuelve la fila completa, por lo que no es necesario seguir avanzando en el flujo. Desde el punto de vista del rendimiento, este es el mejor caso posible, ya que se evita por completo el acceso a disco.

¿Por qué esta fila está en la Row Cache?

Desde mi punto de vista, este es uno de los puntos más importantes del escenario. La Row Cache no almacena datos por ser recientes, sino por ser frecuentemente accedidos. En este caso:

- La fila no está en la Memtable.
- Tampoco se trata de un dato “nuevo”.
- Sin embargo, ha sido leída varias veces en un corto periodo de tiempo.

Esto hace que Cassandra la considere un buen candidato para la Row Cache, optimizando lecturas posteriores.

> Si se produjera una escritura sobre esta fila, la entrada correspondiente en la Row Cache se invalidaría para evitar servir datos inconsistentes.

Mi conclusión para este escenario

Como primer escenario muestra muy bien cómo Cassandra puede resolver una lectura de forma extremadamente eficiente cuando los datos se encuentran en la Row Cache. El flujo de lectura se detiene muy pronto, evitando por completo el acceso a disco y al resto de estructuras internas.

Escenario 2

Row Cache habilitada, pero sin acierto. Lectura desde SSTables

Con este segundo escenario quiero mostrar qué ocurre cuando Cassandra no puede resolver la lectura usando la Row Cache, a pesar de que esté habilitada. Es un caso bastante habitual: la caché existe, pero los datos que se solicitan no han sido accedidos lo suficiente como para estar almacenados en ella.

Este escenario es importante porque representa una situación intermedia: no es el mejor caso (como el Escenario 1), pero tampoco el peor. Cassandra todavía dispone de varias estructuras que le permiten localizar los datos de forma eficiente antes de acceder al disco.

Requisitos previos

De nuevo antes de realizar la lectura, el estado del sistema es el siguiente:

- Existe la tabla `users`.
- La fila con partition key `pk2` y clustering key `ck2` fue escrita hace tiempo y ya se encuentra almacenada en disco.
- Esta fila no ha sido accedida recientemente, por lo que no ha entrado en la Row Cache.
- La Row Cache está habilitada, pero no contiene esta fila.
- La Partition Key Cache está deshabilitada en este escenario.

Además, tras distintos procesos de flush y compaction, los datos se encuentran organizados en una o varias SSTables.

```
SELECT * FROM users WHERE pk = 'pk2' AND ck = 'ck2';
```

¿Dónde están los datos en este momento?

Antes de seguir el flujo, es importante aclarar el contenido de las estructuras:

- **Memtable**

  No contiene `pk2`, ya que la escritura no es reciente y los datos ya han sido volcados a disco.

- **Row Cache (habilitada)**

  No contiene la fila `pk2`, porque no ha sido leída con suficiente frecuencia.

- **Partition Key Cache**

Está deshabilitada, por lo que Cassandra no puede usarla para acelerar la localización de la partición.

- **SSTables**

  Contienen la fila:

  ```
  {
    pk2: {
      ck2: { email: "ismael@example.com" }
    }
  }
  ```

Flujo de lectura

¿Se consulta la Memtable?

Sí, aquí Cassandra siempre comienza comprobando la Memtable para ver si existen datos más recientes.

Resultado: Miss ya que `pk2` no está en memoria , esto indica que no hay escrituras recientes que afecten a esta lectura.

¿Se utiliza la Row Cache?

Sí, ya que está habilitada.

Resultado: Miss, la fila no se encuentra en la Row Cache porque no ha sido accedida con frecuencia y al no poder devolver una fila completa desde caché, Cassandra continúa con el flujo de lectura.

¿Para qué sirve el Bloom Filter en este punto?

Antes de empezar a navegar por las SSTables, Cassandra consulta el Bloom Filter asociado a cada una de ellas.

Resultado: Possible hit, ya que el Bloom Filter indica que la clave `pk2` *podría* estar presente en la SSTable. Esto no garantiza que esté, pero sí evita leer SSTables que con seguridad no contienen esa clave.

> Un Bloom Filter como sabemos puede dar falsos positivos, pero nunca falsos negativos. Es decir, Cassandra podría comprobar una SSTable de más, pero nunca dejaría de comprobar una que realmente contenga los datos.

¿Por qué no se consulta la Partition Key Cache?

En este escenario, la Partition Key Cache está deshabilitada, por lo que Cassandra no dispone de un acceso directo al offset de la partición dentro de la SSTable. Esto obliga a continuar utilizando estructuras en disco para localizar la partición.

¿Cómo se localiza la partición en la SSTable?

Cassandra accede al Partition Summary, una estructura que actúa como un índice resumido de las claves de partición.

```
pk2 -> offset_start = 1000, offset_end = 1500
```

Esto permite a Cassandra acotar la zona del índice donde buscar la partición, evitando recorrerlo completo.

¿Qué papel juega el Partition Index?

Una vez localizada la zona correcta mediante el Partition Summary, Cassandra accede al Partition Index, donde se encuentra el offset exacto de la fila dentro de la partición. Aquí ya se conoce exactamente dónde están los datos dentro del archivo SSTable.

```
pk2:
  ck2 -> disk_offset = 1200
```

¿Por qué se consulta entonces el Compression Offset Map?

Las SSTables suelen estar comprimidas. El **Compression Offset Map** permite traducir el offset lógico a la posición real dentro del archivo comprimido. Gracias a esta estructura, Cassandra puede ir directamente al bloque correcto sin descomprimir el archivo completo.

```
offset 1200 -> uncompressed_position 5000
```

Lectura final desde la SSTable

Con toda la información anterior, Cassandra accede al disco y lee únicamente el bloque necesario de la SSTable, obteniendo finalmente los datos, estos datos se devuelven al cliente.

```
{
  pk2: {
    ck2: { email: "ismael@example.com" }
  }
}
```

```
Cliente -> Nodo Cassandra
  |-> Memtable: Miss
  |-> Row Cache: Miss
  |-> Bloom Filter: Possible hit
  |-> Partition Summary: pk2 -> offset_start=1000
  |-> Partition Index: ck2 -> disk_offset=1200
  |-> Compression Offset Map: 1200 -> pos=5000
  |-> SSTable: Datos leídos
  |-> Resultado devuelto al cliente
```

¿Por qué este escenario es mucho mas costoso que el anterior?

En este caso, Cassandra no ha podido usar la Row Cache para devolver la fila completa. Aunque el flujo sigue estando optimizado, ya es necesario:

- Consultar varias estructuras en disco.
- Acceder físicamente a una SSTable.
- Leer y descomprimir un bloque de datos.

Aun así, gracias al uso del Bloom Filter, el Partition Summary y el Partition Index, Cassandra evita lecturas innecesarias y localiza los datos de forma bastante eficiente.

Mi conclusión para este escenario

Este escenario creo que muestra un caso bastante realista en el que Cassandra debe recorrer gran parte del Read Path Flow para resolver una lectura. Aunque no se dispone de caché para devolver los datos directamente, el sistema sigue optimizando el acceso al disco mediante estructuras auxiliares.

Este flujo sirve como transición hacia el siguiente escenario, donde voy a eliminar todavía más ayudas y Cassandra tendrá que operar en un contexto aún más desfavorable.

Escenario 3

Sin cachés. Lectura completa desde múltiples SSTables

Con este escenario llegamos al caso más desfavorable posible dentro del flujo de lectura de Cassandra. Aquí no hay cachés que ayuden, los datos no están en memoria y, además, están repartidos en varias SSTables debido a escrituras antiguas y procesos de compaction incompletos.

Este es el escenario es el que para mi realmente permite entender por qué Cassandra está diseñada como está, y por qué estructuras como los Bloom Filters o la organización por SSTables son clave para que el sistema siga funcionando bien a gran escala.

Requisitos Previos

De nuevo antes de ejecutar la consulta, el sistema se encuentra en esta situación:

- Existe la tabla `users`.
- La fila con partition key `pk3` y clustering key `ck3` fue escrita hace bastante tiempo.
- Se han producido múltiples escrituras y *flushes*.
- Aún no se ha ejecutado una compaction completa.
- Los datos están repartidos en varias SSTables.
- No existe ningún tipo de caché activa para esta lectura.

Estado de las estructuras:

- Memtable Vacía para `pk3`.
- Row Cache Deshabilitada.
- Partition Key Cache Deshabilitada.
- SSTables Contienen fragmentos de la misma partición.

```
SELECT * FROM users WHERE pk = 'pk3' AND ck = 'ck3';
```

¿Qué problema plantea este escenario?

Aquí Cassandra se enfrenta a varias dificultades al mismo tiempo:

- No puede devolver la fila desde memoria.
- No sabe de antemano en qué SSTable está la versión más reciente del dato.
- Debe tener en cuenta posibles tombstones.
- Puede haber versiones antiguas del mismo dato en distintas SSTables.

Esto obliga a Cassandra a leer, comparar y reconciliar datos antes de devolver un resultado correcto.

Flujo de lectura

¿Se consulta la Memtable?

Sí, siempre es el primer paso.

Resultado: Miss, ya que no hay datos recientes en memoria relacionados con `pk3`.

¿Se consulta alguna caché?

No, ya que en este escenario la Row Cache está deshabilitada, la Partition Key Cache está deshabilitada y Cassandra no puede saltarse ningún paso del flujo de lectura.

¿Qué papel juegan los Bloom Filters ahora?

Cassandra consulta el **Bloom Filter de cada SSTable**.

Resultado: SSTable 1 Possible hit, SSTable 2 Possible hit, SSTable 3 Possible hit

Esto indica que todas podrían contener la clave `pk3`, por lo que ninguna puede descartarse.

> aquí se ve el peor caso realista. Cassandra tiene que revisar varias SSTables, pero sigue evitando recorrer aquellas que con certeza no contienen la clave.

¿Cómo se localiza la partición en cada SSTable?

Para cada SSTable candidata, Cassandra repite el mismo proceso, se consulta el Partition Summary para acotar la zona del índice y accede al Partition Index para encontrar el offset exacto de `pk3`.

```
SSTable 1 -> pk3 offset 800
SSTable 2 -> pk3 offset 1200
SSTable 3 -> pk3 offset 600
```

¿Qué ocurre con la compresión?

Cada SSTable puede tener distintos bloques comprimidos donde Cassandra utiliza el Compression Offset Map de cada SSTable para traducir los offsets lógicos a posiciones reales en disco y leer únicamente los bloques necesarios.

Lectura desde múltiples SSTables

Cassandra lee los datos relevantes de todas las SSTables implicadas, incluyendo versiones antiguas del dato, versiones más recientes y tombstones que indiquen eliminaciones.

```
SSTable 1 -> ck3 = email_old
SSTable 2 -> ck3 = email_new
SSTable 3 -> tombstone ck2
```

¿Cómo decide Cassandra qué dato es válido?

Aquí entra en juego el proceso de reconciliación, donde se comparan timestamps, se descartan versiones antiguas, se respetan los tombstones y se aplica la política de consistencia configurada. El resultado final es la versión más reciente y válida del dato.

```
{
  pk3: {
    ck3: { email: "final@example.com" }
  }
}
```

```
Cliente -> Nodo Cassandra
  |-> Memtable: Miss
  |-> Cachés: No disponibles
  |-> Bloom Filters (varias SSTables): Possible hit
  |-> Partition Summary (cada SSTable)
  |-> Partition Index (cada SSTable)
  |-> Compression Offset Map
  |-> Lectura de múltiples SSTables
  |-> Reconciliación de datos
  |-> Resultado final al cliente
```

¿Por qué este es el peor caso?

Porque Cassandra se ve obligada a acceder a disco varias veces, leer datos redundantes, gestionar tombstones y comparar múltiples versiones del mismo dato. Aun así, el sistema sigue siendo funcional y predecible, incluso bajo estas condiciones.

Mi conclusión para este escenario

Este escenario demuestra que Cassandra está diseñada para degradar de forma controlada. Cuando no hay cachés y los datos están fragmentados, el coste de la lectura aumenta, pero el

sistema sigue garantizando la consistencia y la corrección de los resultados.

Este es el escenario que mejor explica por qué Cassandra sacrifica simplicidad interna a cambio de escalabilidad y fiabilidad.